PERSONOVA — JARVIS

DETAILED 0→100 COMPLETION SPECIFICATION + CLAUDE HANDOFF

Purpose: Give this document to the developer and Claude together with the existing unfinished Personova/JARVIS repository. The instruction is to continue the existing work, preserve the current frontend, fix the prototype defects, and take the project to a tested 100% completion state.

1. PROJECT VISION

Personova/JARVIS is a futuristic Windows-oriented AI assistant. The user must be able to type or speak naturally, ask ordinary questions, receive ChatGPT-like answers on the page, optionally hear those answers through voice, and request safe computer/productivity actions.

Target architecture: Voice/Text → React → Flask API → NLP/Intent → Context + Memory → Planner/Policy → Validated Tools → AI/Retrieval/Windows/Vision/Productivity → Result → React → Optional TTS.

2. CURRENT FOUNDATION — ALREADY PRESENT

- React/Vite/TypeScript frontend and routing.
- Futuristic JARVIS dashboard/assistant visual design, avatar, holographic UI, animations and assets.
- Startup screen and JARVIS overlay.
- Assistant page with browser SpeechRecognition and SpeechSynthesis.
- Notes, Calendar and Tasks using LocalStorage.
- Flask backend with CORS.
- parser.py, step_splitter.py and planner.py rule-based foundation.
- command_handler.py with browser/Windows automation foundations.
- file_control.py with file/folder operations.
- vision.py with screenshot/OpenCV/Tesseract OCR foundation.
- memory.py with basic JSON memory.

3. CURRENT STATUS

The project is a substantial prototype, not a finished assistant. The frontend foundation should be preserved. The largest missing capability is genuine natural-language conversation: the current brain is rule-based rather than ChatGPT-like. NLP, safe planning, reliable tools, memory/context, robust voice lifecycle, testing and production-quality error handling also remain.

4. NON-NEGOTIABLE FRONTEND RULE

- Do NOT rebuild the frontend from scratch.
- Preserve the existing JARVIS theme, avatar, routes, assets, animations, Notes, Calendar and Tasks.
- Enhance existing Assistant components instead of replacing the product design.
- Keep LocalStorage compatibility or add migration.
- React must never directly control Windows.

5. ASSISTANT — FINAL REQUIREMENTS

- Typed input and Enter-to-send.
- Microphone start/stop with permission handling.
- Central voice state machine: IDLE, LISTENING, PROCESSING, SPEAKING, ERROR, CONFIRMING.
- Conversation history with user/JARVIS roles.
- Markdown and code rendering.
- Loading/processing indicator.
- Stop-speaking control.

- Retry/offline/error UI.
- Optional voice output.
- Short-term conversation context for follow-ups.
- Structured action and confirmation cards.
- Configurable backend URL.

6. KNOWN FRONTEND DEFECTS TO FIX

- SpeechRecognition lifecycle can use stale state and restart repeatedly.
- Recognition errors can create retry loops.
- Multiple recognition instances can overlap without centralized control.
- Navigation may occur from raw recognized text even if backend execution fails.
- Startup and overlay can both speak, causing duplicate speech.
- No proper assistant state machine or backend-offline state.

7. PRODUCTIVITY INTEGRATION

- Notes: natural-language create, read, search, edit and delete.
- Tasks: create, list, edit, complete, reopen and delete; parse due date/priority.
- Calendar: create, list, edit and delete; parse dates/times safely.
- Ambiguous dates/times must trigger clarification.
- Protect LocalStorage against malformed JSON.
- Prevent duplicate spoken reminders.

8. BACKEND — REQUIRED EVOLUTION

Current prototype flow is approximately React → POST /command → parser → splitter → planner → command/brain → reply. Replace the string-only architecture with structured request/response handling while reusing useful modules.

- Add /api/health.
- Use structured JSON for conversation, action, confirmation and error responses.
- Replace global last_command suppression with request/session-aware deduplication.
- Centralize exception handling and logging.
- Add timeouts/cancellation where appropriate.
- Move AI provider, Tesseract path and other settings to environment configuration.
- Never expose secrets to React.

9. NLP — COMPLETE SPECIFICATION

Every request should become a structured representation such as: **{ intent, confidence, entities, parameters, steps, requires_confirmation, context_requirements }**.

- Conversation questions must not require command keywords.
- Extract entities such as application, file, date, time, task title and search query.
- Resolve references such as 'that', 'it', 'tomorrow' and 'the first result' using context.
- Low confidence must trigger clarification, not a random action.
- Multi-action requests must become ordered structured steps.

10. CRITICAL CURRENT BUG — STEP SPLITTER

The current splitter naively splits on the substring 'and'. This can corrupt valid words such as 'calendar'. Replace it with token-aware splitting or, preferably, let the NLP/planner produce structured multi-step plans.

11. PLANNER — FINAL ROLE

- Classify conversation, retrieval, productivity, vision, browser, application and file intents.
- Extract/validate parameters.
- Use context for follow-ups.
- Create ordered steps.
- Assign confidence and risk.
- Require confirmation for destructive/high-risk actions.
- Call only allowlisted tools.
- Ask clarification when necessary.

12. BRAIN / CHATGPT-LIKE CONVERSATION

The current brain.py contains predefined responses for a small set of phrases and cannot answer arbitrary questions. Replace/extend it with a real conversational AI provider layer.

- General questions and explanations.
- Coding/technical help.
- Writing and summarization.
- Contextual follow-up questions.
- Markdown/UI-ready answers.
- Optional TTS.
- Provider abstraction so the AI service can be changed independently.
- API keys only through secure environment configuration; never hard-code them.

13. COMMAND HANDLER — REQUIRED FIXES

- Copy operation must actually copy the file and verify destination.
- Permanent deletion must require confirmation; prefer recycle-bin behavior where practical.
- Move operations need structured source/destination parameters.
- Do not assume browser focus.
- OCR clicking must never fall back to an unconditional click when OCR matching fails.
- Typing must safely handle special/Unicode characters.
- Return structured execution results and meaningful errors.
- Never claim success unless the executor confirms success.

14. FILE CONTROL

- Safe search within intended directories.
- Exact/fuzzy matching distinction.
- Ask when multiple files match.
- Validate source/destination paths.
- Correct create/copy/move/rename/delete behavior.
- Explicit overwrite handling.
- Confirmation for destructive operations.
- Structured success/failure metadata.

15. VISION / OCR

- Make Tesseract path configurable.

- Verify Tesseract during startup/health checks.
- Use on-demand or throttled OCR.
- Return OCR text and bounding boxes/confidence where useful.
- Support read-screen, find-text and click-text.
- Use confidence/ambiguity checks.
- Return diagnostics when screenshot capture fails.
- The previously observed screen-grab failure must become an acceptance test.

16. MEMORY

- Separate short-term conversation context from persistent memory.
- Persistent memory should be explicit/user-approved.
- Support preference, personal fact, project context and instruction categories.
- Safe update/delete and corruption recovery.
- Do not permanently save every conversation automatically.
- Use recent context for follow-up questions.

17. ALLOWLISTED TOOL REGISTRY

- conversation_answer
- web_search / web_retrieve
- open_application
- browser_search / browser_click / browser_type
- screen_capture / ocr_screen / click_ocr_text
- file_search / file_open / file_create / file_copy / file_move / file_rename / file_delete
- note_create/read/search/update/delete
- task_create/list/update/complete/delete
- calendar_create/list/update/delete
- memory_save/recall/update/delete

Each tool must have a defined input schema, validation, risk level, executor and structured result. The AI must never directly execute arbitrary shell commands.

18. CURRENT-INFORMATION / WEB LAYER

Provide a separate retrieval route for questions requiring current information. Keep retrieval independent from the conversation provider. Retrieved information must not bypass tool validation.

19. API CONTRACT

Move beyond a single reply string. A conceptual response is: **{ success, message, response_type, action, requires_confirmation, data, request_id }**. Suggested response types: conversation, action, confirmation, error.

20. VOICE ARCHITECTURE

Use one centralized controller: IDLE → LISTENING → PROCESSING → SPEAKING → IDLE, with ERROR and CONFIRMING branches. Only one recognizer may run at a time. Use controlled retries/backoff. Provide stop listening and stop speaking. Speak only final user-facing responses.

21. SECURITY

- Never commit API keys/secrets.
- Never execute arbitrary shell commands from model output.
- Use an allowlisted tool system.

- Validate every tool argument.
- Confirm permanent deletion and other high-risk operations.
- Validate file paths and ambiguous targets.
- Use timeouts/cancellation.
- Log diagnostics without secrets.
- Never report an action as successful without executor confirmation.

22. CLEAN SETUP

- Add requirements.txt or pyproject configuration.
- Document clean Python virtual-environment setup.
- Document Node/npm setup.
- Document Tesseract installation/configuration.
- Document AI provider environment variables.
- Document frontend/backend URL configuration.
- Do not require the bundled/archive venv.

23. TESTING STRATEGY

- NLP unit tests.
- Planner tests for simple and multi-step requests.
- File tests using temporary directories.
- Safety/confirmation tests.
- Memory tests including malformed JSON.
- Vision/OCR controlled tests.
- API integration tests.
- Frontend assistant-state tests.
- End-to-end text-to-response tests.
- Manual Windows acceptance tests.

24. 0→100 IMPLEMENTATION ROADMAP

- Phase 0 — Audit every file: COMPLETE/PARTIAL/BROKEN/MISSING.
- Phase 1 — Stabilize existing frontend/backend.
- Phase 2 — API/configuration and clean dependencies.
- Phase 3 — NLP intent/entity/confidence/context.
- Phase 4 — Structured safe planner.
- Phase 5 — Allowlisted validated tool registry.
- Phase 6 — Reliable Windows/file/browser actions.
- Phase 7 — Reliable screen capture/OCR.
- Phase 8 — ChatGPT-like conversational AI.
- Phase 9 — Memory and conversation context.
- Phase 10 — Stable voice input/output.
- Phase 11 — Natural-language Notes/Tasks/Calendar.
- Phase 12 — Current-information retrieval.
- Phase 13 — Logging, retries, timeouts and error recovery.
- Phase 14 — Testing and UI polish.

- Phase 15 — Final acceptance and documentation.

25. ACCEPTANCE TESTS

- 'Hello JARVIS' → natural greeting; no OS action.
- 'What is polymorphism?' → useful AI answer on page; optional TTS.
- 'Explain that more simply' → previous context is used.
- 'Open Chrome' → Chrome actually opens/activates and result is confirmed.
- 'Search for C++ tutorials' → search executes and reports result.
- 'Search Python tutorials and open the first result' → ordered safe multi-step execution.
- 'Create folder College' → folder exists afterward.
- 'Copy report.pdf to Documents' → destination exists afterward.
- 'Delete report.pdf' → confirmation appears first.
- 'Read my screen' → screen capture + OCR returns useful text.
- 'Click visible Submit text' → OCR match verified before clicking.
- 'Remember that I prefer C++' → explicit memory saved.
- 'What language do I prefer?' → memory retrieved.
- 'Add a JARVIS task tomorrow' → correct task/date created.
- Microphone denied → clear error without loop.
- Backend offline → clear retry/error state.
- AI provider unavailable → graceful failure, UI stays usable.
- Ambiguous file → ask user to choose.

26. FINAL 100% CHECKLIST

- Frontend preserved and functional.
- Typed input works.
- Voice input works reliably.
- Voice output works reliably.
- General natural-language Q&A; works.
- Contextual follow-ups work.
- NLP extraction works.
- Low-confidence clarification works.
- Multi-step planning works.
- Allowlisted tools work.
- Windows actions only execute through backend.
- Browser actions are verified.
- Copy really copies.
- Delete requires confirmation.
- Screen capture works.
- OCR works.
- OCR click is confidence-aware.
- Notes integrate with JARVIS.
- Tasks integrate with JARVIS.
- Calendar integrates with JARVIS.

- Explicit memory works.
- Current-information retrieval works where required.
- Structured API responses work.
- Secrets are protected.
- Clean setup works.
- Automated tests exist.
- Windows acceptance tests pass.
- Documentation is complete.

27. MASTER CLAUDE HANDOFF PROMPT

You are the lead engineer taking over the supplied unfinished Personova JARVIS repository. Inspect the entire repository first. Do NOT rebuild from scratch and do NOT replace the existing JARVIS frontend unnecessarily. Preserve the current design, assets, routes, animations, Notes, Calendar, Tasks and useful backend foundations. Classify every component as COMPLETE, PARTIAL, BROKEN or MISSING. Then implement this specification phase by phase.

The final product must support typed and spoken natural-language input, genuine NLP intent/entity/context understanding, safe multi-step planning, ChatGPT-like conversational answers shown in the existing UI, optional spoken answers, explicit persistent memory, Notes/Tasks/Calendar integration, screen capture/OCR, supported Windows/browser/file automation, current-information retrieval where appropriate, structured API responses, confirmations, logging, timeouts, retries and comprehensive tests.

Fix the known defects: naive 'and' splitting, defective copy behavior, unsafe deletion, hard-coded configuration, fragile voice lifecycle, global duplicate suppression, canned conversational brain and weak structured errors. Never execute arbitrary shell commands from LLM output. Use validated allowlisted tools. Never claim success without executor confirmation.

Reuse working code wherever practical. Run unit, integration, end-to-end and Windows acceptance tests. Fix failures before declaring completion. At the end report changed files, setup steps, environment variables, tests, known limitations and final acceptance status. Do not declare 100% complete until the acceptance checklist passes.

28. DEFINITION OF DONE

The project is complete only when a fresh developer can set it up from documentation, start frontend/backend, interact by voice or text, ask arbitrary questions and receive useful page/voice answers, ask contextual follow-ups, perform safe supported computer actions, use file/browser/vision functions, control Notes/Tasks/Calendar naturally, explicitly store and recall memory, retrieve current information when needed, recover cleanly from failures and pass the final acceptance suite.

Final principle: Do not add random features just to make the project bigger. Transform the existing Personova foundation into a reliable, intelligent, safe, testable and genuinely usable JARVIS assistant.